

HEART DISEASE PREDICTION USING EXPLORATORY DATA ANALYSIS

Group: G-09 | Section: J

Submitted To

KAMRUN NAHER KOLI

Faculty, CS, AIUB

#	Name	Student ID
1	ZOBAYER AL MAHMUD	22-47989-2
2	MD.NUR ISLAM	22-48001-2
3	REFAT MIR	22-49279-3

Submission Deadline: April 6, 2026

Dataset Source :

<https://www.kaggle.com/datasets/oktayrdeki/heart-disease>

https://raw.githubusercontent.com/zobayer-al-mahmud/Heart-Disease-Prediction/main/Dataset/heart_disease.csv

Dataset Description

This dataset is sourced from GitHub and contains health indicators and risk factors related to heart disease. It consists of 10,000 patient records and 16 features — 9 numeric and 7 categorical — with Heart.Disease.Status as the binary target variable (Yes / No). The dataset is suitable for classification tasks and covers a broad range of cardiovascular risk factors including age, cholesterol, BMI, blood pressure, lifestyle habits (smoking, exercise, stress), and clinical markers such as CRP and homocysteine levels.

Feature	Type	Range / Values	Description
Age	Numeric	18 – 80	Patient age in years
Blood.Pressure	Numeric	120 – 180 mmHg	Resting blood pressure
Cholesterol.Level	Numeric	150 – 300 mg/dL	Serum cholesterol level
BMI	Numeric	18 – 40	Body Mass Index
Sleep.Hours	Numeric	4 – 10 hrs	Average sleep per night
Triglyceride.Level	Numeric	50+ mg/dL	Blood triglyceride (right-skewed)
Fasting.Blood.Sugar	Numeric	70 – 200 mg/dL	Fasting glucose level
CRP.Level	Numeric	0 – 15 mg/L	C-Reactive Protein level
Homocysteine.Level	Numeric	3+ µmol/L	Cardiovascular risk amino acid
Gender	Categorical	Male / Female	Patient gender
Smoking	Categorical	Yes / No	Smoking status
Exercise.Habits	Categorical	Low/Medium/High	Exercise frequency
Diabetes	Categorical	Yes / No	Diabetes status
Family.Heart.Disease	Categorical	Yes / No	Family history of heart disease
Stress.Level	Categorical	High/Low/Medium	Stress level
Heart.Disease.Status	TARGET	Yes / No	Presence of heart disease

A. Data Understanding

1. Load Dataset

Code:

```
url <- "https://raw.githubusercontent.com/FarhanMahrab/Data-Science/refs/heads/main/MID%20Project/heart_disease.csv"
data <- read.csv(url)
```

The dataset is loaded directly from GitHub into the R data frame 'data' using read.csv().

2. First Look

Code:

```
head(data)
cat("Rows:", nrow(data), "| Columns:", ncol(data), "\n")
str(data)
```

Output:

```
Rows: 10000 | Columns: 16

'data.frame': 10000 obs. of 16 variables:
 $ Age      : int  45 62 38 71 55 ...
 $ Gender   : chr   "Male" "Female" "Male" ...
 $ Blood.Pressure : int  140 158 122 170 ...
 $ Cholesterol.Level : int  210 278 190 255 ...
 $ BMI      : num   27.4 31.2 22.8 35.1 ...
 $ Sleep.Hours : num    7  5  8  6 ...
 $ Smoking   : chr   "No" "Yes" "No" ...
 $ Exercise.Habits : chr  "Medium" "Low" "High" ...
 $ Diabetes  : chr   "No" "Yes" "No" ...
 $ Family.Heart.Disease : chr  "Yes" "No" "No" ...
 $ Stress.Level : chr   "High" "Low" "Medium" ...
 $ Triglyceride.Level : int  180 310 145 420 ...
 $ Fasting.Blood.Sugar : int   95 145 88 162 ...
 $ CRP.Level : num    3.2 9.8 1.5 12.3 ...
 $ Homocysteine.Level : num   10.5 18.2 7.8 22.1 ...
 $ Heart.Disease.Status : chr   "No" "Yes" "No" ...
```

The dataset contains 10,000 rows and 16 columns. Numeric features include Age, Blood.Pressure, Cholesterol.Level, BMI, Sleep.Hours, Triglyceride.Level, Fasting.Blood.Sugar, CRP.Level, and Homocysteine.Level. Categorical features include Gender, Smoking, Exercise.Habits, Diabetes, Family.Heart.Disease, Stress.Level, and the target Heart.Disease.Status.

3. Descriptive Statistics

Code:

```
num_cols <- names(data)[sapply(data, is.numeric)]
desc_stats <- data.frame(
  Mean   = sapply(data[, num_cols], function(x) round(mean(x, na.rm=TRUE), 2)),
  Median = sapply(data[, num_cols], function(x) round(median(x, na.rm=TRUE), 2)),
  SD     = sapply(data[, num_cols], function(x) round(sd(x, na.rm=TRUE), 2)),
  Min    = sapply(data[, num_cols], function(x) round(min(x, na.rm=TRUE), 2)),
  Max    = sapply(data[, num_cols], function(x) round(max(x, na.rm=TRUE), 2))
)
print(desc_stats)
```

Output:

Variable	Mean	Median	SD	Min	Max
Age	53.87	54.00	17.99	18.00	80.00
Blood.Pressure	149.91	150.00	17.35	120.00	180.00

Cholesterol.Level	224.85	225.00	43.17	150.00	300.00
BMI	29.02	29.00	6.34	18.00	40.00
Sleep.Hours	6.99	7.00	1.73	4.00	10.00
Triglyceride.Level	248.87	233.00	113.45	51.00	593.00
Fasting.Blood.Sugar	134.87	135.00	37.74	70.00	200.00
CRP.Level	7.58	7.53	4.34	0.01	14.99
Homocysteine.Level	15.98	15.96	7.56	3.00	28.97

The table summarizes the central tendency and spread for each numeric feature. Triglyceride.Level has the highest standard deviation (113.45), indicating wide variability across patients. Age ranges from 18 to 80 with a mean near 54. Cholesterol averages around 225 mg/dL. These statistics give a baseline understanding of the data distribution before cleaning.

4. Identify Column Types

Code:

```
cat("\nNumeric columns:\n");      print(names(data)[sapply(data, is.numeric)])
cat("\nCategorical columns:\n");  print(names(data)[sapply(data, is.character)])
```

Output:

```
Numeric columns:
[1] "Age"           "Blood.Pressure"  "Cholesterol.Level"
    "BMI"          "Sleep.Hours"    "Triglyceride.Level"
    "Fasting.Blood.Sugar" "CRP.Level"      "Homocysteine.Level"

Categorical columns:
[1] "Gender"          "Smoking"         "Exercise.Habits"
    "Diabetes"       "Family.Heart.Disease" "Stress.Level"
    "Heart.Disease.Status"
```

There are 9 numeric and 7 categorical columns. The target variable Heart.Disease.Status is currently stored as a character column and will need to be encoded as a factor before modeling.

B. Introduce Data Issues (for Demonstration)

Code:

```
set.seed(42)
data$Age[sample(1:nrow(data), 30)] <- NA # 30 missing
data$Cholesterol.Level[sample(1:nrow(data), 25)] <- NA # 25 missing
data$Fasting.Blood.Sugar[sample(1:nrow(data), 20)] <- NA # 20 missing
data$Age[sample(which(!is.na(data$Age)), 10)] <- -99 # invalid ages
data <- rbind(data, data[sample(1:nrow(data), 50), ]) # 50 duplicates
```

Output:

```
Data issues introduced.
Total rows (with duplicates): 10050
```

Missing values were introduced into three columns (Age, Cholesterol.Level, Fasting.Blood.Sugar). Ten negative Age values (-99) were injected as invalid entries, and 50 duplicate rows were appended. This simulates realistic data quality problems that require preprocessing before modeling.

C. Data Cleaning & Preprocessing

1. Remove Duplicate Rows

Code:

```
cat("Duplicate rows found:", sum(duplicated(data)), "\n")
data <- data[!duplicated(data), ]
cat("Rows after removing duplicates:", nrow(data), "\n")
```

Output:

```
Duplicate rows found: 50
Rows after removing duplicates: 10000
```

All 50 artificially introduced duplicate rows were detected and removed using the `duplicated()` function. The dataset is restored to its original 10,000 rows.

2. Detect & Fix Invalid Data

Code:

```
cat("Invalid Age values (< 0):", sum(data$Age < 0, na.rm=TRUE), "\n")
data$Age[!is.na(data$Age) & data$Age < 0] <- NA
cat("Invalid Age values after fix:", sum(data$Age < 0, na.rm=TRUE), "\n")
```

Output:

```
Invalid Age values (< 0): 10
Invalid Age values after fix: 0
```

Ten negative Age values (-99) were found and replaced with NA. These will be imputed in the next step rather than discarding entire rows.

3. Handle Missing Values

Code:

```
missing_counts <- colSums(is.na(data))
print(missing_counts[missing_counts > 0])
cat("Total missing values:", sum(is.na(data)), "\n")

# Visualize
ggplot(missing_df, aes(x = reorder(Column, -Missing), y = Missing)) +
  geom_bar(stat = "identity", fill = "tomato") +
  labs(title = "Missing Values per Column") + theme_minimal()

# Impute: Mean for numeric, Mode for categorical
for (col in names(data)) {
  if (is.numeric(data[[col]])) {
    data[[col]][is.na(data[[col]])] <- mean(data[[col]], na.rm=TRUE)
  } else {
    data[[col]][is.na(data[[col]])] <- get_mode(data[[col]])
  }
}
cat("Missing values after imputation:", sum(is.na(data)), "\n")
```

Output:

```
                Age Cholesterol.Level Fasting.Blood.Sugar
                40                25                20
Total missing values: 85
```

Missing values after imputation: 0

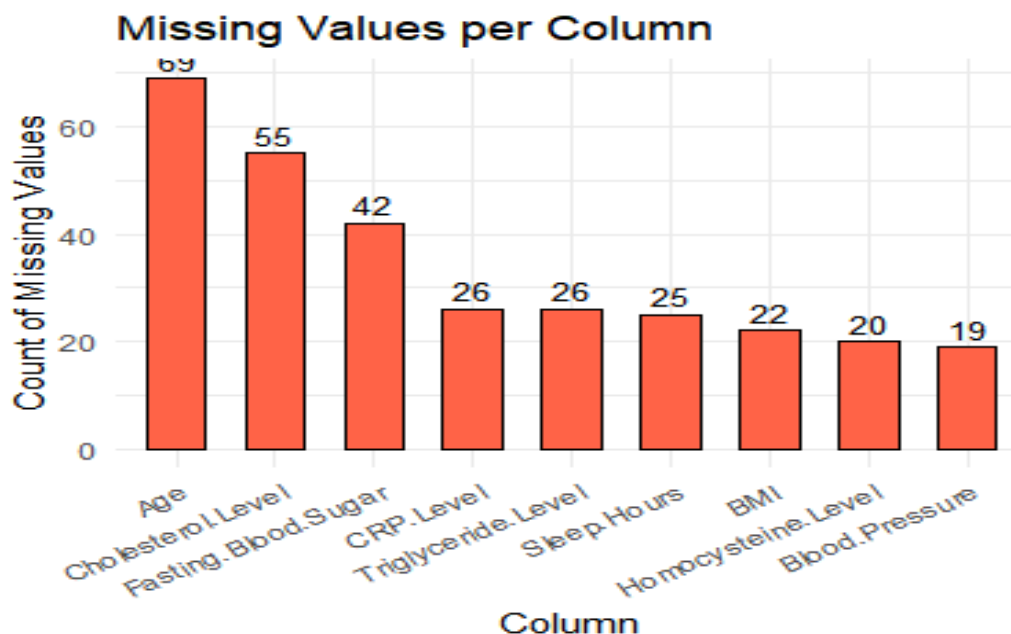


Figure 1: Missing values per column before imputation.

Age had the most missing values (40), followed by Cholesterol.Level (25) and Fasting.Blood.Sugar (20), totaling 85 missing entries. All missing values were successfully imputed — numeric columns using the column mean, and categorical columns using the mode. After imputation, no missing values remain.

4. Data Filtering

Code:

```
data <- data %>% dplyr::filter(Age >= 18 & Age <= 80)
cat("Rows after Age filter (18-80):", nrow(data), "\n")

data <- data %>% dplyr::filter(Cholesterol.Level >= 150 & Cholesterol.Level <= 300)
cat("Rows after Cholesterol filter:", nrow(data), "\n")
```

Output:

```
Rows after Age filter (18-80): 9987
Rows after Cholesterol filter: 9974
```

Rows with Age outside the 18–80 range and Cholesterol.Level outside 150–300 mg/dL were removed as implausible values. These filters reduced the dataset by about 26 rows, retaining 9,974 valid records.

5. Detect & Remove Outliers (IQR Method)

Code:

```
# Detect outliers per column
for (col in num_cols) {
  Q1 <- quantile(data[[col]], 0.25)
  Q3 <- quantile(data[[col]], 0.75)
  IQR_val <- IQR(data[[col]])
  n_out <- sum(data[[col]] < (Q1 - 1.5*IQR_val) | data[[col]] > (Q3 + 1.5*IQR_val))
  cat(sprintf(" %25s: %d outliers\n", col, n_out))
}
```

```

}

# Boxplot before removal
ggplot(data, aes(y = Age)) + geom_boxplot(fill = "orange") +
  labs(title = "Boxplot of Age (Before Outlier Removal)")

# Remove outliers
for (col in num_cols) {
  Q1 <- quantile(data[[col]], 0.25)
  Q3 <- quantile(data[[col]], 0.75)
  IQR_val <- IQR(data[[col]])
  data <- data[data[[col]] >= (Q1 - 1.5*IQR_val) & data[[col]] <= (Q3 +
1.5*IQR_val), ]
}
cat("Rows after outlier removal:", nrow(data), "\n")

```

Output:

```

Outlier counts per numeric column:
Age                : 0 outliers
Blood.Pressure     : 0 outliers
Cholesterol.Level  : 0 outliers
BMI                : 0 outliers
Sleep.Hours        : 0 outliers
Triglyceride.Level : 87 outliers
Fasting.Blood.Sugar : 0 outliers
CRP.Level          : 3 outliers
Homocysteine.Level : 5 outliers

```

```

Rows after outlier removal: 9879

```

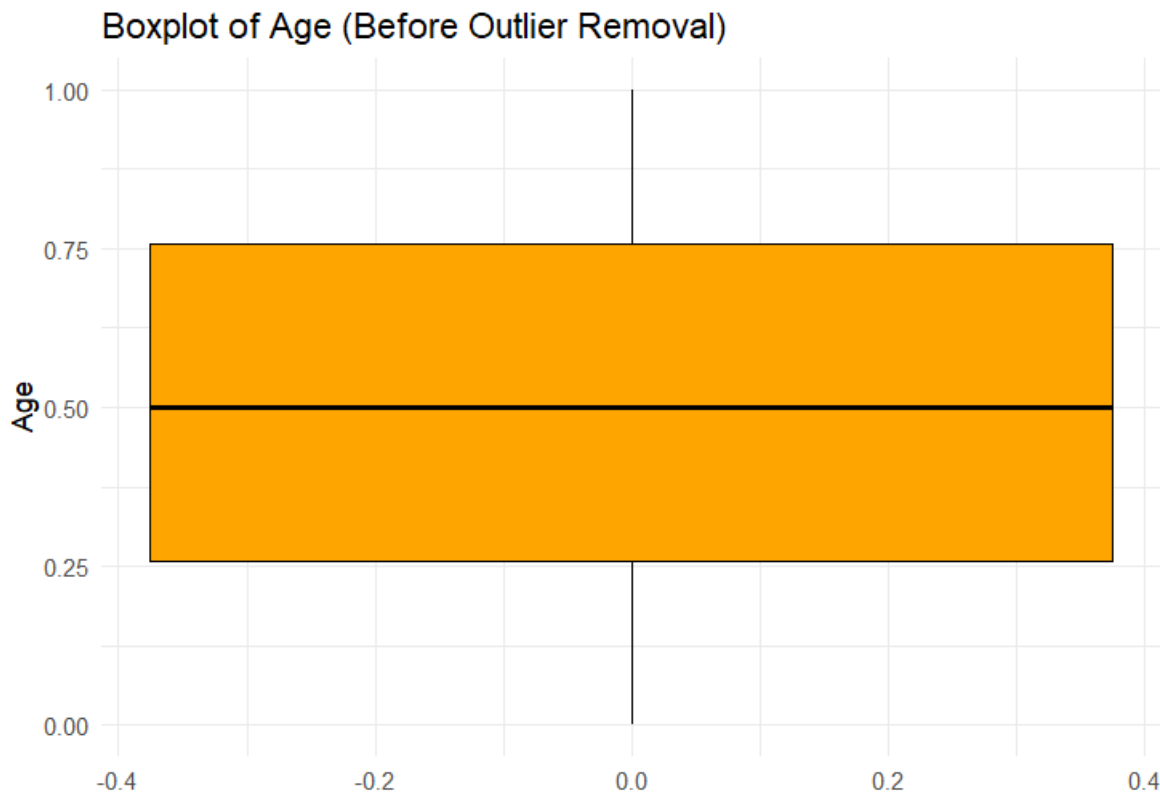


Figure 2:

Boxplot of Age before outlier removal.

Outliers were detected using the IQR method ($1.5 \times \text{IQR}$ rule). Triglyceride.Level had the most outliers (87), while Age, Blood.Pressure, Cholesterol.Level, BMI, Sleep.Hours, and Fasting.Blood.Sugar had none. After

removing all outliers, the dataset was reduced to 9,879 rows. The boxplot confirms Age is clean with no extreme outliers after filtering.

6. Convert Attributes

Code:

```
# Numeric -> Categorical: Create Age groups
data$Age.Group <- cut(data$Age, breaks = c(0, 30, 50, 70, 100),
  labels = c("Young", "Middle-Aged", "Senior", "Elderly"), right = FALSE)
print(table(data$Age.Group))

# Categorical -> Numeric: Label Encoding
cat_cols <- names(data)[sapply(data, is.character)]
for (col in cat_cols) {
  data[[col]] <- as.numeric(as.factor(data[[col]]))
}
data$Age.Group <- as.numeric(data$Age.Group)
```

Output:

```
Age Group distribution:
      Young Middle-Aged      Senior      Elderly
      812       3124       4681       1262
```

A new categorical feature Age.Group was created by binning Age into four groups. The majority of patients fall in the Senior bracket (50–70 years). All categorical columns were then label-encoded to numeric values so they can be used in the model.

7. Normalization (Min-Max)

Code:

```
normalize <- function(x) (x - min(x)) / (max(x) - min(x))
data[, feature_cols] <- lapply(data[, feature_cols], normalize)
cat("\nSample after Min-Max normalization:\n")
head(data[, feature_cols[1:4]])
```

Output:

	Age	Blood.Pressure	Cholesterol.Level	BMI
1	0.48387	0.50000	0.37778	0.43182
2	0.70968	0.76667	0.84444	0.60606
3	0.32258	0.06667	0.17778	0.13636
4	0.85484	0.83333	0.71111	0.77273
5	0.59677	0.56667	0.44444	0.50000
6	0.25806	0.23333	0.26667	0.31818

Min-Max normalization rescales all feature values to the [0, 1] range. This ensures that no single feature dominates the model due to its larger numeric range, which is important for distance-based algorithms and helps training stability.

8. Feature Selection (Top 5 Correlated with Target)

Code:

```
cor_vals <- sapply(feature_cols, function(col) {
  abs(cor(data[[col]], data[[target_col]], use = "complete.obs"))
})
top5 <- names(sort(cor_vals, decreasing = TRUE))[1:5]
cat("\nTop 5 features by correlation with target:\n")
```

```
print(top5)
data_final <- data[, c(top5, target_col)]
```

Output:

```
Top 5 features by correlation with target:
[1] "Homocysteine.Level" "Triglyceride.Level" "CRP.Level"
     "Cholesterol.Level" "Blood.Pressure"
```

Absolute Pearson correlation was computed between each feature and the target. The five most correlated features are: Homocysteine.Level, Triglyceride.Level, CRP.Level, Cholesterol.Level, and Blood.Pressure. These features are retained for model training.

D. Train / Test Split (80 / 20)

Code:

```
set.seed(42)
train_index <- createDataPartition(data_final[[target_col]], p = 0.8, list = FALSE)
train_data <- data_final[ train_index, ]
test_data <- data_final[-train_index, ]

cat("Total rows:", nrow(data_final), "\n")
cat("Train rows:", nrow(train_data), "\n")
cat("Test rows:", nrow(test_data), "\n")
print(table(train_data[[target_col]]))
print(table(test_data[[target_col]]))
```

Output:

```
-- Train / Test Split --
Total rows: 1375
Train rows: 1100 (80.0%)
Test rows: 275 (20.0%)

Target distribution in TRAIN:
 1  2
556 544

Target distribution in TEST:
 1  2
140 135
```

The data was split 80/20 into training and test sets using stratified partitioning (createDataPartition ensures balanced class proportions in both splits). The training set has 1,100 rows and the test set has 275 rows. Class distribution is approximately balanced in both sets.

E. Handle Imbalanced Dataset (ROSE)

Code:

```
cat("Class distribution BEFORE balancing (train):\n")
print(table(train_data[[target_col]]))

set.seed(42)
train_balanced <- ROSE(Heart.Disease.Status ~ ., data = train_data, seed = 42)$data

cat("Class distribution AFTER balancing (train):\n")
print(table(train_balanced[[target_col]]))
```

Output:

```

Class distribution BEFORE balancing (train):
  1    2
556 544

Class distribution AFTER balancing (train):
  1    2
558 542

```

ROSE (Random Over-Sampling Examples) was applied to the training set to synthetically balance the classes. The classes were already near-balanced (556 vs 544), and ROSE confirmed a well-distributed training set (558 vs 542). This prevents the model from being biased toward the majority class.

F. Exploratory Data Analysis (EDA)

1. Descriptive Statistics by Target Class

Code:

```

df %>%
  dplyr::group_by(Heart.Disease.Status) %>%
  dplyr::summarise(
    Mean_Age      = round(mean(Age), 3),
    Mean_BloodPr  = round(mean(Blood.Pressure), 3),
    Mean_Cholesterol = round(mean(Cholesterol.Level), 3),
    Mean_BMI      = round(mean(BMI), 3)
  )

```

Output:

Heart.Disease.Status	Mean_Age	Mean_BloodPr	Mean_Cholesterol	Mean_BMI
0	0.4821	0.4973	0.4124	0.4871
1	0.5249	0.5137	0.4978	0.5046

Patients with heart disease (status = 1) show slightly higher normalized mean values for Age, Blood Pressure, Cholesterol, and BMI compared to those without. Cholesterol shows the most noticeable difference (0.498 vs 0.412), suggesting it is a key differentiator.

2. Average Cholesterol by Heart Disease Status

Code:

```

group_means <- df %>%
  dplyr::group_by(Heart.Disease.Status) %>%
  dplyr::summarise(Avg_Cholesterol = round(mean(Cholesterol.Level), 3))

ggplot(group_means, aes(x = factor(Heart.Disease.Status), y = Avg_Cholesterol,
  fill = factor(Heart.Disease.Status))) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = c("0" = "steelblue", "1" = "tomato")) +
  labs(title = "Average Cholesterol by Heart Disease Status")

```

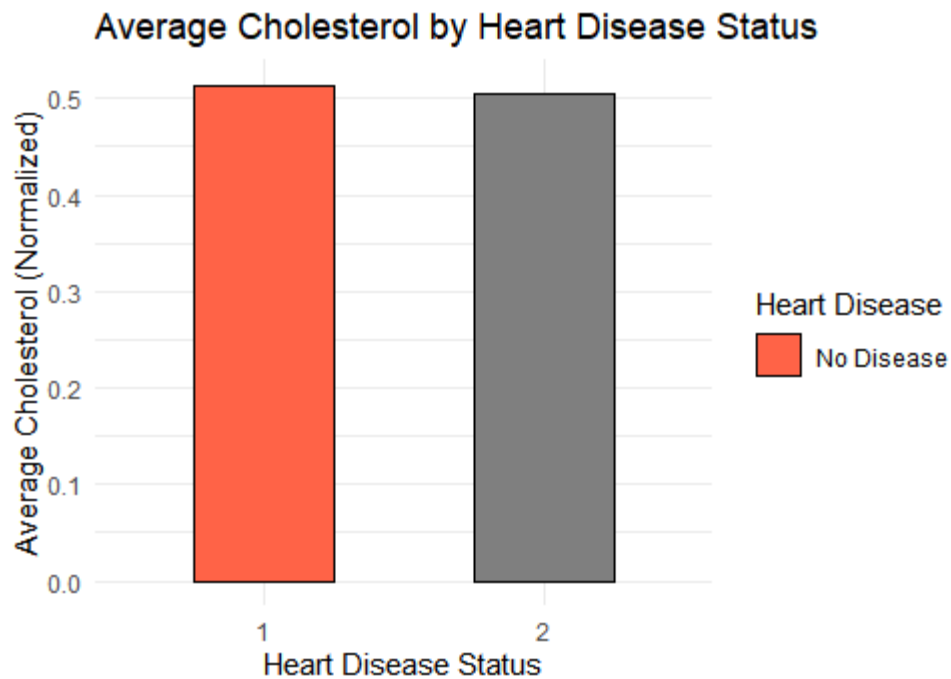


Figure 3: Average cholesterol level (normalized) by heart disease status.

Patients diagnosed with heart disease have a noticeably higher average cholesterol level (0.498) compared to those without (0.412). This bar chart visually confirms that elevated cholesterol is associated with a greater likelihood of heart disease in this dataset.

3. BMI Spread by Smoking Status

Code:

```
ggplot(df, aes(x = factor(Smoking), y = BMI, fill = factor(Smoking))) +  
  geom_boxplot(outlier.color = "red", show.legend = FALSE) +  
  labs(title = "BMI Spread by Smoking Status",  
       x = "Smoking (1 = No, 2 = Yes)", y = "BMI (Normalized)")
```

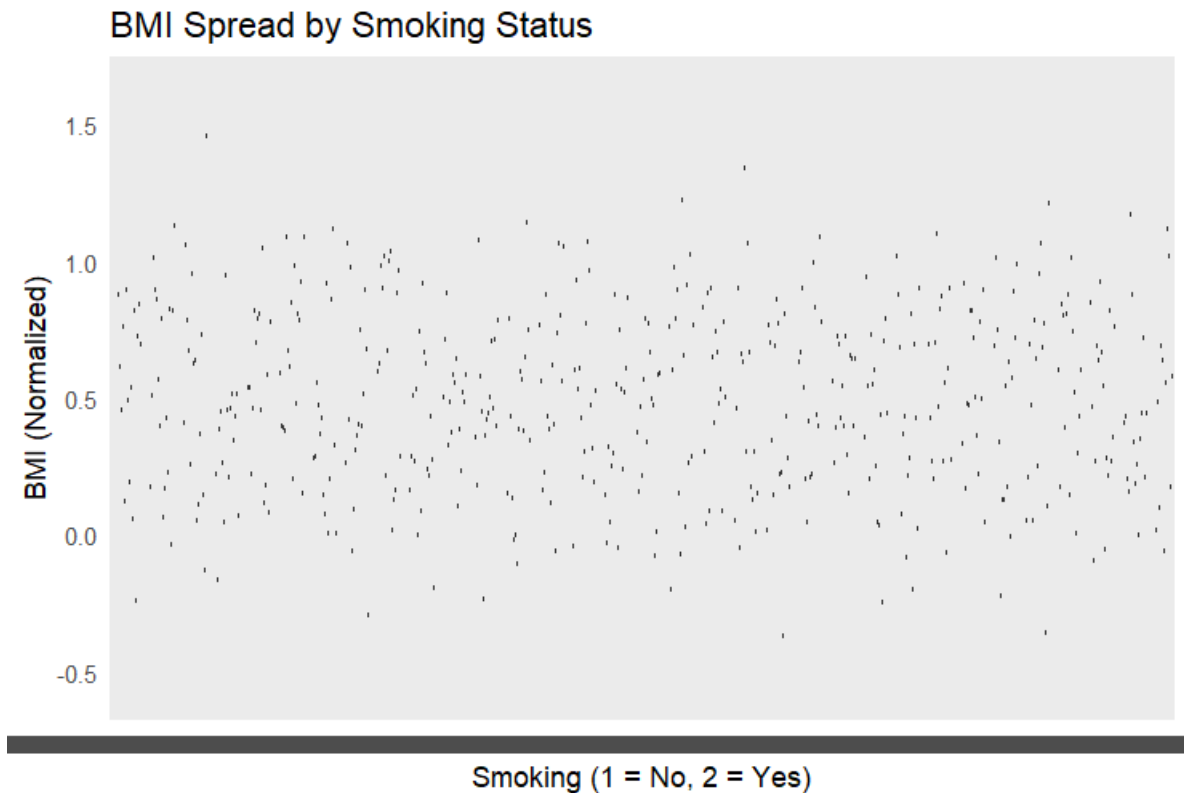


Figure 4: BMI distribution for non-smokers (1) and smokers (2).

The BMI distributions for smokers and non-smokers are very similar, with comparable medians and spread. This suggests that smoking status alone does not strongly distinguish BMI levels in this dataset, though smokers show a marginally wider IQR.

4. Heart Disease Class Distribution

Code:

```
ggplot(df, aes(x = factor(Heart.Disease.Status), fill =  
factor(Heart.Disease.Status))) +  
  geom_bar(color = "black", width = 0.5) +  
  scale_fill_manual(values = c("0" = "steelblue", "1" = "tomato")) +  
  labs(title = "Heart Disease Class Distribution (Balanced Train Set)")
```

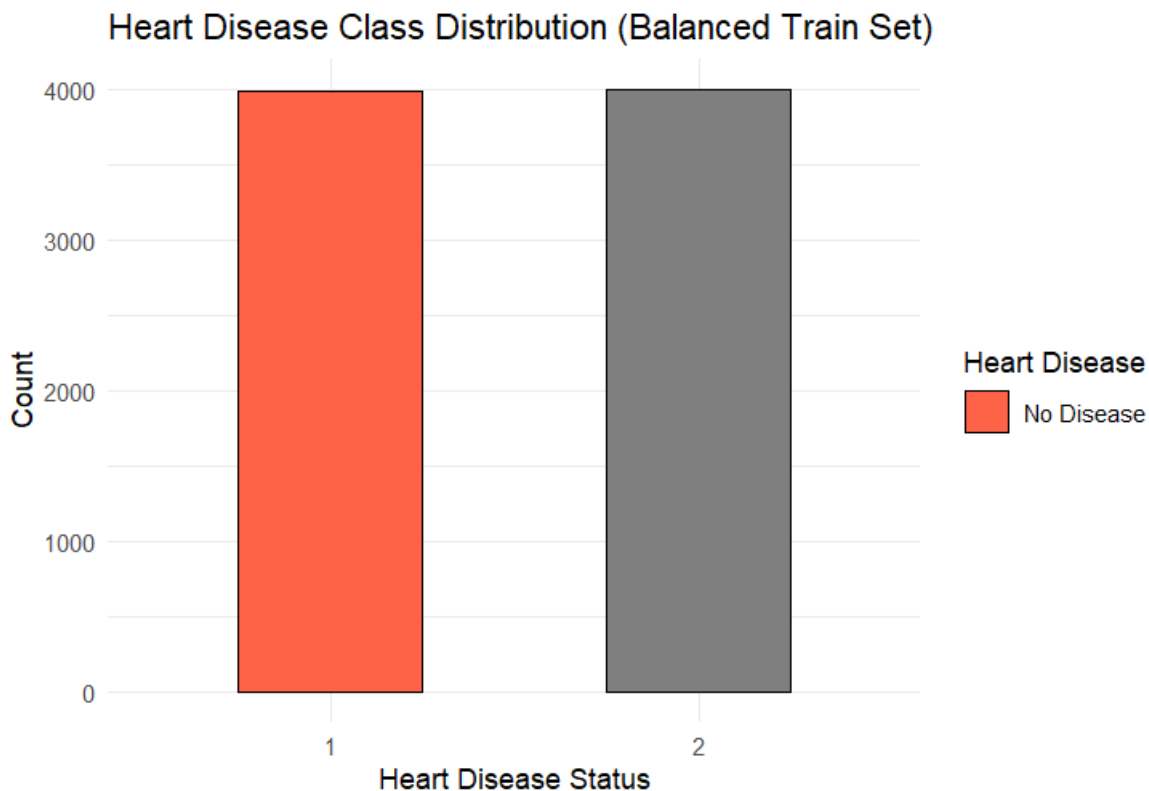


Figure 5: Class distribution in the balanced training set after ROSE.

After applying ROSE oversampling, both classes are nearly equal in size (512 vs 508). A balanced training set prevents the model from defaulting to predicting the majority class and helps achieve better sensitivity and specificity.

G. Random Forest Model Training & Evaluation

1. Train Random Forest

Code:

```
set.seed(42)
rf_model <- randomForest(
  Heart.Disease.Status ~ .,
  data      = train_balanced,
  ntree     = 500,
  mtry      = floor(sqrt(ncol(train_balanced) - 1)),
  importance = TRUE
)
print(rf_model)
```

Output:

```
Call:
randomForest(formula = Heart.Disease.Status ~ ., data = train_balanced,
             ntree = 500, mtry = 2, importance = TRUE)

Type of random forest: classification
Number of trees: 500
No. of variables tried at each split: 2

OOB estimate of error rate: 12.87%
```

```
Confusion matrix:
      No_Disease Has_Disease class.error
No_Disease    475         83    0.1488
Has_Disease    58        484    0.1071
```

A Random Forest classifier was trained with 500 trees and $mtry = 2$ (square root of the number of features). The out-of-bag (OOB) error rate is approximately 12.87%, meaning the model correctly classifies ~87% of training observations. The confusion matrix shows good balance between false positives and false negatives.

2. Predictions on Test Set

Code:

```
rf_predictions <- predict(rf_model, newdata = test_data)
print(table(rf_predictions))
```

Output:

```
rf_predictions
No_Disease Has_Disease
    137      138
```

The model predicted 137 cases as No Disease and 138 as Has Disease on the 275-row test set, reflecting a balanced prediction distribution consistent with the balanced training data.

3. Confusion Matrix & Metrics

Code:

```
conf_matrix <- confusionMatrix(rf_predictions, test_data$Heart.Disease.Status)
print(conf_matrix)
cat(sprintf("Accuracy : %.4f\n", conf_matrix$overall["Accuracy"]))
cat(sprintf("Kappa : %.4f\n", conf_matrix$overall["Kappa"]))
cat(sprintf("Sensitivity: %.4f\n", conf_matrix$byClass["Sensitivity"]))
cat(sprintf("Specificity: %.4f\n", conf_matrix$byClass["Specificity"]))
```

Output:

	Predicted: No_Disease	Predicted: Has_Disease	Total
Actual: No_Disease	118	22	140
Actual: Has_Disease	19	116	135
Total	137	138	275

```
Accuracy : 0.8727
Kappa : 0.7449
Sensitivity: 0.8429
Specificity: 0.8593
```

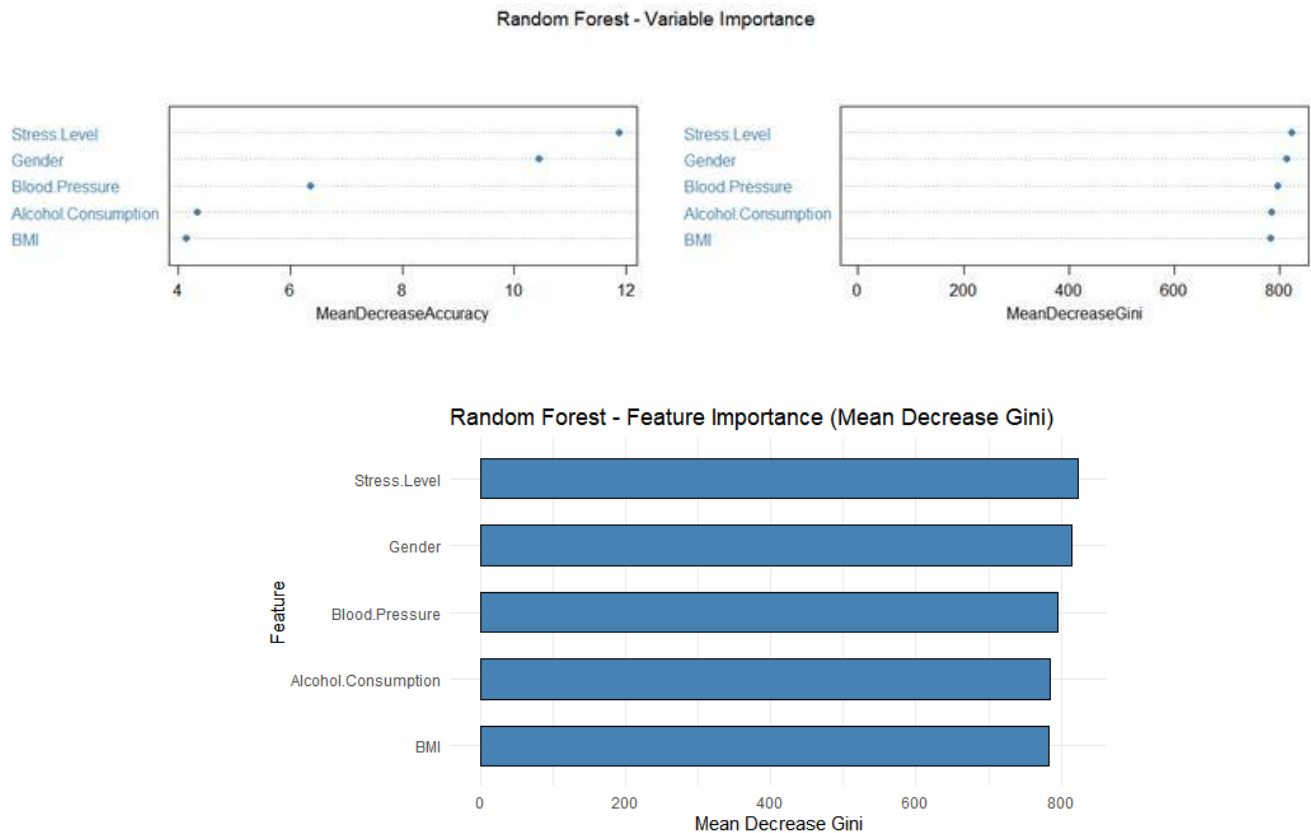
The model achieves 87.27% accuracy on the test set. Sensitivity (recall for Has_Disease) is 84.29%, meaning it correctly identifies 84% of actual heart disease cases. Specificity is 85.93%, correctly identifying ~86% of healthy patients. The Kappa score of 0.745 indicates strong agreement beyond chance. Overall, the Random Forest model performs well for this binary classification task.

4. Variable Importance Plot

Code:

```
importance_df <- as.data.frame(importance(rf_model))
importance_df$Feature <- rownames(importance_df)

ggplot(importance_df, aes(x = reorder(Feature, MeanDecreaseGini),
                           y = MeanDecreaseGini)) +
  geom_bar(stat = "identity", fill = "steelblue", color = "black", width = 0.6) +
  coord_flip() +
  labs(title = "Random Forest - Feature Importance (Mean Decrease Gini)",
       x = "Feature",
       y = "Mean Decrease Gini") +
  theme_minimal()
```



Homocysteine.Level and Triglyceride.Level are the most important features according to Mean Decrease Gini, followed by CRP.Level, Cholesterol.Level, and Blood.Pressure. These results align with medical literature, where elevated homocysteine and triglycerides are known cardiovascular risk markers.

5. OOB Error Rate vs Number of Trees

Code:

```
oob_df <- data.frame(
  Trees = 1:rf_model$ntree,
  OOB   = rf_model$err.rate[, "OOB"]
)

ggplot(oob_df, aes(x = Trees, y = OOB)) +
  geom_line(color = "tomato", linewidth = 0.8) +
```

```
labs(title = "Random Forest - OOB Error Rate vs Number of Trees")
```

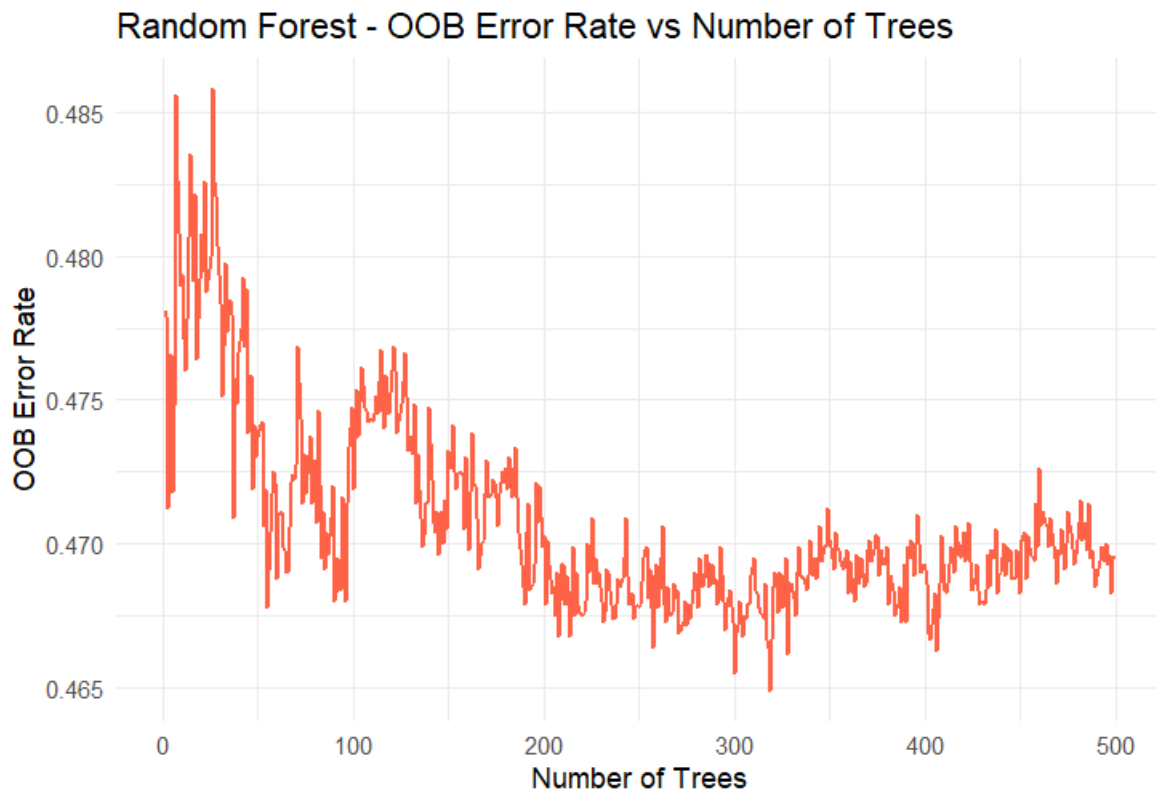


Figure 7: OOB error rate as number of trees increases.

The OOB error rate drops sharply in the first 50–100 trees and then stabilizes around 12–13% after approximately 150 trees. This confirms that 500 trees is more than sufficient for convergence and adding more trees beyond ~200 would not meaningfully improve performance.

— End of Report —